

# PoRE Lab2 实验报告

- 姓名：徐锦云
- 学号：23307130447
- 耗时：12 小时

## 一、提交内容

- Lab2 实验报告
- 填充后的源代码文件：Lab2\_Task\_23307130447.cpp

## 二、Task 1：访问 class 中私有成员变量（30%）

### 1. 私有成员变量访问代码

对应类定义（题目给定）：

```
class lab2Class_23307130447 {
public :
    explicit lab2Class_23307130447(int number) {
        this->x = number;
    }
private :
    short var_a;
    long long var_b;
    char var_c;
    short var_d;
    short var_e;
    long long var_f;
    int x; // lab2_task1 访问变量 x
    int var_g;
    char var_h;
    long long var_i;
    char var_j;
};
```

我们应该先画个图（以字节为单位）：

|       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|
| var_a | var_a | /     | /     | /     | /     | /     | /     |
| var_b | var_b | var_b | var_b | var_b | var_b | var_b | var_b |
| var_c | /     | var_d | var_d | var_e | var_e | /     | /     |
| var_f | var_f | var_f | var_f | var_f | var_f | var_f | var_f |

下面就轮到4字节的x了，也就是我们的目标。它在对应的第32字节上。

在 main 函数中，通过偏移量访问私有成员 x 的代码如下（节选）：

```
// Lab2 Task1 访问x变量的值指针
// Write your code here...
// short 2;char 1;long long 8;int 4;
// 因此x相对于对象起始地址的偏移量为32字节。
char* base_task1 = reinterpret_cast<char*>(&obj_task1);
int* x_ptr = reinterpret_cast<int*>(base_task1 + 32);
```

2. 偏移量计算过程和说明

实验环境为 64 位 Linux + g++ 编译器，遵循常见的对齐规则：

- short 按 2 字节对齐
- int 按 4 字节对齐
- long long 按 8 字节对齐
- char 按 1 字节对齐
- 每一个参数都是起始地址是其对应字节数的倍数，其余位置空出来。

因此，再次印证刚才画的图是正确的。我们在这里放一个完整的偏移图。

| var_a | var_a | /     | /     | /     | /     | /     | /     |
|-------|-------|-------|-------|-------|-------|-------|-------|
| var_b | var_b | var_b | var_b | var_b | var_b | var_b | var_b |
| var_c | /     | var_d | var_d | var_e | var_e | /     | /     |
| var_f | var_f | var_f | var_f | var_f | var_f | var_f | var_f |
| x     | x     | x     | x     | var_g | var_g | var_g | var_g |
| var_h | /     | /     | /     | /     | /     | /     | /     |
| var_i | var_i | var_i | var_i | var_i | var_i | var_i | var_i |
| var_j |       |       |       |       |       |       |       |

3.成功访问运行截图

```
g++ Lab2_Task_23307130447.cpp -std=c++11 -O0 -g
./a.out 1234

# pore @ pore in ~/pore26/pore_23307130447/Lab2 on git:master x [8:26:03]
$ ./a.out 1234
ID=23307130447 Lab2 Task1 x = 1234
```

这一问解决了。

## 三、Task 2: 调用 class 中私有虚函数 (30%)

### 1. 私有虚函数访问代码

对应类定义:

```
class lab2Class_23307130447_Func {
public :
    explicit lab2Class_23307130447_Func(int number) {
        this->x = number;
    }
private :
    int x;
    virtual int func_A() { return x + 100; }
    virtual void func_B() { x = x * 0; }
    int func_C() { return x + 3; }
    void func_D() { x = x * 0; }
    virtual int func_E() { return x + 10000; }
    virtual int getValue() { return x + 1; } // lab2_task2 调用函数
    virtual long long func_F() { return (long long)(x + 1000000); }
    int func_G() { return x + 2; }
    virtual void func_H() { x = x * 0; }
};
```

在 main 中通过虚函数表调用私有虚函数 `getValue` 的代码为:

```
lab2Class_23307130447_Func obj_task2(passwd);

// Lab2 Task2 调用私有虚函数
// func_A(0), func_B(1), func_E(2), getValue(3), func_F(4), func_H(5)。
// 通过对象首地址处的vptr取得虚函数表指针, 再通过下标3拿到getValue的函数指针并调用。
using GetValueFunc = int (*)(void*);
void** vtable_task2 = *reinterpret_cast<void***>(&obj_task2);
GetValueFunc getValuePtr = reinterpret_cast<GetValueFunc>(vtable_task2[3]);
int value_x = getValuePtr(&obj_task2);
```

### 2. 虚函数表大致内容

在该类中, 声明了以下几个虚函数 (按顺序) :

1. `virtual int func_A()`
2. `virtual void func_B()`
3. `virtual int func_E()`
4. `virtual int getValue()`
5. `virtual long long func_F()`
6. `virtual void func_H()`

根据 Itanium C++ ABI 的常见实现:

- 对于没有虚析构的简单类, 虚函数表中通常按照**声明顺序**依次存放各虚函数的入口地址

- 因此可以认为虚表中的槽位大致为：

| 槽下标 | 函数名      | 函数类型             |
|-----|----------|------------------|
| 0   | func_A   | int (this)       |
| 1   | func_B   | void (this)      |
| 2   | func_E   | int (this)       |
| 3   | getValue | int (this)       |
| 4   | func_F   | long long (this) |
| 5   | func_H   | void (this)      |

由于是在 64 位系统上运行，这里每个函数对应的指针的大小都是 8 个字节。由目标导向，我们需要利用虚函数访问函数 `getValue`，并且求它的返回值。于是类似于第一个 task，这里我们直接定义一个指针来指向虚函数表的表头，就能够访问 `getValue` 函数，然后访问返回值即可。

因此，只要拿到对象首地址处的 `vptr`，即可取得虚表首地址 `vtable_task2`，再通过 `vtable_task2[3]` 取得 `getValue` 的地址，强制转换为 `int(*) (void*)` 类型并传入对象地址即可成功调用。

### 3. 成功调用运行截图

```
g++ Lab2_Task_23307130447.cpp -std=c++11 -O0 -g
./a.out 1234
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab2 on git:master x [8:26:03]
$ ./a.out 1234
ID=23307130447 Lab2 Task1 x = 1234
ID=23307130447 Lab2 Task2 value = 1235
```

此问解决。

## 四、Task 3：继承后子类的私有变量访问与虚函数表（40%）

### 1. 私有成员变量访问代码

子类定义如下：

```
class lab2Class_23307130447_Derived : public lab2Class_23307130447_Func {
public :
    explicit lab2Class_23307130447_Derived(int number) : lab2Class_23307130447_Func(number)
{
    int reversed = 0;
    while (number != 0) {
        int digit = number % 10;
        reversed = reversed * 10 + digit;
        number /= 10;
    }
}
```

```

        this->s.y = reversed;
    }
private :
    char var_da;
    short var_db;
    long long var_dc;
    int var_i;
    struct {
        short var_de;
        char var_df;
        int y; // lab2_task3 访问变量 y
    } s;
    virtual int func_A() { return s.y + 200; }
    virtual void func_dB() { s.y = s.y * 0; }
    virtual int getNewValue() { return s.y + 2; } // lab2_task3 调用函数
    long long var_dg;
    char var_dh;
};

```

这个代码的public部分定义了一个 lab2Class\_Derived 函数，输入信息为一个 int 类型的数字，并且这个函数调用了基类中的 lab2Class\_Func 函数。随后就是整个函数的核心部分：首先定义了一个 int 类型的变量 reversed 并且将初值设为 0，然后进行一个循环，当输入的值不为 0 时，取这个值的个位，并存入变量 digit，然后将这个值赋给 reversed，接着将原先的 number 除以 10（自然是取整数了），开始循环。最后的结果就是 s.y 会指向 reversed 中的信息，且这个值是原先 number 的反转值。简言之，输入“321”，s.y 指向“123”。

在 main 中通过偏移量访问子类中的私有成员 s.y 的代码为：

```

//父类要先算
//再看struct,它占8字节，这就是一个特殊结构体
// short 2;char 1;long long 8;int 4;
//经过计算，得知y的位置在地址32
char* base_task3 = reinterpret_cast<char*>(&obj_task3);
int* y_ptr = reinterpret_cast<int*>(base_task3 + 32);
int y = *y_ptr;

```

## 2. 私有成员变量偏移量计算过程和说明

首先计算父类子对象（lab2Class\_23307130447\_Func）的大小与布局：

它由两个部分组成：对象开头有一个 vptr（虚函数表指针），在 64 位下占 8 字节，偏移区间 [0, 7]；接着是 int x，需要 4 字节对齐。

再看struct，未来它是一个特殊的“单位”，遵循之前提到的规则：

|        |        |        |   |   |   |   |   |
|--------|--------|--------|---|---|---|---|---|
| var_de | var_de | var_df | / | y | y | y | y |
|--------|--------|--------|---|---|---|---|---|

一共占8字节。

根据上面的推理以及源代码，把完整的图画出：

| vp <sub>ptr</sub> | vp <sub>ptr</sub> | vp <sub>ptr</sub> | vp <sub>ptr</sub> | vp <sub>ptr</sub> | vp <sub>ptr</sub> | vp <sub>ptr</sub> | vp <sub>ptr</sub> |
|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|
| x                 | x                 | x                 | x                 | var <sub>da</sub> | /                 | var <sub>db</sub> | var <sub>db</sub> |
| var <sub>dc</sub> | var <sub>dc</sub> | var <sub>dc</sub> | var <sub>dc</sub> | var <sub>dc</sub> | var <sub>dc</sub> | var <sub>dc</sub> | var <sub>dc</sub> |
| var <sub>i</sub>  | var <sub>i</sub>  | var <sub>i</sub>  | var <sub>i</sub>  | var <sub>de</sub> | var <sub>de</sub> | var <sub>df</sub> | /                 |
| y                 | y                 | y                 | y                 |                   |                   |                   |                   |

也就是说，需求的y是从地址32开始的。由此，代码就能写了。

### 3. 私有虚函数访问代码

在 `main` 中通过虚函数表调用 `getNewValue` 的实现代码为：

```
// 虚函数表中先保留父类的虚函数槽位（有的被覆盖），顺序为：
// func_A(被子类覆盖，占槽0)、func_B(1)、func_E(2)、getValue(3)、func_F(4)、func_H(5)，
// 子类新增虚函数依次追加：func_dB(6)、getNewValue(7)。
// 因此通过槽位7取得getNewValue并调用。
using GetNewValueFunc = int (*)(void*);
void** vtable_task3 = *reinterpret_cast<void***>(&obj_task3);
GetNewValueFunc getNewValuePtr = reinterpret_cast<GetNewValueFunc>(vtable_task3[7]);
int value_y = getNewValuePtr(&obj_task3);
```

### 4. 虚函数表大致内容和分析说明

子类中新增的私有虚函数为：

```
virtual int func_A() { return s.y + 200; }
virtual void func_dB() { s.y = s.y * 0; }
virtual int getNewValue() { return s.y + 2; } // lab2_task3 调用函数
```

父类 `lab2Class_23307130447_Func` 中的虚函数顺序为：

1. `func_A`
2. `func_B`
3. `func_E`
4. `getValue`
5. `func_F`
6. `func_H`

子类 `lab2Class_23307130447_Derived`：

- 继承自父类，并覆盖了 `func_A`：

```
virtual int func_A() { return s.y + 200; }
```

- 新增两个虚函数：

```
virtual void func_dB() { s.y = s.y * 0; }
virtual int getNewValue() { return s.y + 2; }
```

- 子类会复用父类虚函数表的布局：
  - 槽 0: `func_A` (被子类覆盖, 指向 `Derived::func_A`)
  - 槽 1: `func_B` (仍为父类实现)
  - 槽 2: `func_E` (父类)
  - 槽 3: `getValue` (父类)
  - 槽 4: `func_F` (父类)
  - 槽 5: `func_H` (父类)
- 子类新增虚函数会被追加到虚表末尾, 顺序与声明顺序一致:
  - 槽 6: `func_dB`
  - 槽 7: `getNewValue`

因此, 通过 `vtable_task3[7]` 取得的就是 `getNewValue` 的入口地址, 强制转换为 `int(*) (void*)` 并传入对象地址即可完成对**私有虚函数**的调用。

## 5. 成功运行截图

```
g++ Lab2_Task_23307130447.cpp -std=c++11 -O0 -g
./a.out 1234
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab2 on git:master x [9:10:19] C:134
● $ ./a.out 1234
ID=23307130447 Lab2 Task1 x = 1234
ID=23307130447 Lab2 Task2 value = 1235
ID=23307130447 Lab2 Task3 y = 4321
ID=23307130447 Lab2 Task3 value = 4323
```

到此, 该任务结束。

---

## 五、实验分析与总结

### 1. 关于偏移量的推导与验证

- 本次实验中, 完全基于 **对齐规则** 和 **类型大小** 从头推导各个成员的偏移量, 并用指针运算访问私有成员。
- 若有需要, 也可以通过 `sizeof`/手动打印地址、或 `offsetof` 等方式进行验证。
- 对于带虚函数的类, 对象首地址处会多出一个 `vp`tr, 需要把这个指针的大小也计入偏移计算。

### 2. 汇编代码中的信息

- 对带虚函数的类, 编译后的汇编/反汇编中可以看到:
  - 每个虚函数的实现地址;
  - 编译器生成的虚表 (包含函数指针的数组);

- 构造函数中为 `vptr` 赋值的代码。
- 对于普通成员变量，访问时通常表现为：
  - `mov` 指令使用 `this + offset` 的形式访问成员，其中 `offset` 即我们推导的偏移量。

这些信息都可以用来交叉验证我们手工推导的内存布局。

### 3. 不同架构下的差异

- 不同架构（如 32 位 vs 64 位、不同 ABI）中：
  - 指针大小不同（4 字节 vs 8 字节），会直接影响对象首部 `vptr` 的大小；
  - 某些平台上 `long` 的大小也会不同（4 字节或 8 字节）；
  - 对齐规则可能存在细微差异。
- 因此，同一份 C++ 源码在不同平台上的偏移量和对象整体大小都可能不同，访问私有成员时必须以**当前编译平台**为准。

### 4. 多继承对偏移量与虚函数表的影响

- 本实验中只涉及**单继承**，此时对象内存布局大致为：
  - 先是父类子对象（含 `vptr` 和父类成员）；
  - 再是子类自己的成员。
- 若为**多继承**，常见实现会为每个有虚函数的基类维护一个独立的 `vptr`，派生类对象中可能会出现多个 `vptr`，甚至采用“虚基类表”等更复杂的结构：
  - 每个基类子对象有独立的起始偏移；
  - 各自拥有自己的虚函数表视图；
  - 通过不同类型的指针（`Base1*` / `Base2*`）指向同一个派生类对象时，`this` 指针往往会有不同的偏移调整。
- 因此，多继承下的偏移量与虚函数表分析会复杂许多，需要结合编译器的具体 ABI 说明来推导。